

The Bus Factor Protocol

Succession Governance for Open-Source Package Registries

P-RICK RESEARCH PROGRAM

an independent working-paper series on missing software

working paper P-010 v1.0 • series III

October 7, 2026 • cc by 4.0

Abstract — Every package registry on earth — npm, PyPI, crates.io, Maven Central, RubyGems — governs *code identity* with mature machinery: names, namespaces, provenance attestations, signing, 2FA, reproducible builds. None governs *maintainership continuity* at all. Whether a package has one exhausted maintainer or five active ones is invisible to the registry, unverifiable by dependents, and unimprovable by anyone — so the ecosystem’s load-bearing packages decay into abandonment by default, and abandonment is where every catastrophic governance failure of the last decade actually began. The 2024 xz-utils backdoor was not primarily a security failure; it was a *succession* failure — a burned-out maintainer transferred control, over years, to a helpful stranger who turned out to be an attacker. left-pad (2016) removed 11% of the npm ecosystem’s reachable graph in hours; event-stream (2018) hijacked an abandoned package to attack cryptocurrency wallets; colors and faker (2022) showed a live maintainer can weaponize his own packages at will; node-ipc (2022) showed protestware. The pattern across incidents: registries treat maintainer state as out of scope, so the most important state in the supply chain transitions through private messages between strangers. We specify the **bus factor protocol**: an abandonment state machine observable by the registry; a **succession policy as code** declared per package (quorum, dormancy thresholds, takeover rules); a **dormancy escalation ladder** ending in a narrow *security-hold* (new publishes freeze, installs never do); a **trust-elevation ladder** for new maintainers (delay-locked, co-signed early releases, mandatory review windows) designed specifically to make the xz attack structurally hard; a **dead-hand transfer** instrument for maintainers who want their packages to outlive their attention; and a public, append-only **continuity ledger** of every succession event, so that “who maintains this” becomes an auditable, historical fact rather than a folk belief. Every component exists in production somewhere — threshold publishing, attested releases, timelock ceremonies, registry policy enforcement; the composition does not exist anywhere. The gap is **STRONG**: the failure mode is decade-frequent and occasionally catastrophic, the affected population is effectively all software, and the only incumbents are cultural norms (“looking for maintainer” issue templates) that have provably not scaled.

p-rick working paper P-010 · series III (continuity) · draft 1.0

Every package registry on earth — npm, PyPI, crates.io, Maven Central, RubyGems — governs *code identity* with mature machinery: names, namespaces, provenance attestations, signing, 2FA, reproducible builds. None governs *maintainership continuity* at all. Whether a package has one exhausted maintainer or five active ones is invisible to the registry, unverifiable by dependents, and unimprovable by anyone — so the ecosystem’s load-bearing packages decay into abandonment by default, and abandonment is where every catastrophic governance failure of the last decade actually began. The 2024 xz-utils backdoor was not primarily a security failure; it was a *succession* failure — a burned-out maintainer transferred control, over years, to a helpful stranger who turned out to be an attacker. left-pad (2016) removed 11% of the npm ecosystem’s reachable graph in hours; event-stream (2018) hijacked an abandoned package to attack cryptocurrency wallets; colors and faker (2022) showed a live maintainer can weaponize his own packages at will; node-ipc (2022) showed protestware. The pattern across incidents: registries treat maintainer state as out of scope, so the most important state in the supply chain transitions through private messages between strangers. We specify the **bus factor protocol**: an abandonment state machine observable by the

registry; a **succession policy as code** declared per package (quorum, dormancy thresholds, takeover rules); a **dormancy escalation ladder** ending in a narrow *security-hold* (new publishes freeze, installs never do); a **trust-elevation ladder** for new maintainers (delay-locked, co-signed early releases, mandatory review windows) designed specifically to make the xz attack structurally hard; a **dead-hand transfer** instrument for maintainers who want their packages to outlive their attention; and a public, append-only **continuity ledger** of every succession event, so that “who maintains this” becomes an auditable, historical fact rather than a folk belief. Every component exists in production somewhere — threshold publishing, attested releases, timelock ceremonies, registry policy enforcement; the composition does not exist anywhere. The gap is **STRONG**: the failure mode is decade-frequent and occasionally catastrophic, the affected population is effectively all software, and the only incumbents are cultural norms (“looking for maintainer” issue templates) that have provably not scaled.

1 Introduction

In March 2024, the world learned that a compression library at the base of nearly every Linux distribution had been backdoored. The mechanics, reconstructed by the community over the following weeks, were not an exploit of code or cryptography. The attacker spent years becoming useful — submitting patches, taking over maintenance burdens, reducing the load on a single exhausted maintainer — until he *was* the maintainer, then shipped a backdoor through the ordinary release process, signed with the ordinary keys, passing the ordinary reviews. The postmortems classified it as a supply-chain compromise, which is true the way “the Titanic had a hull breach” is true. The operative failure was upstream of every technical control: **xz had a succession process, and the succession process was “the tired guy hands the keys to whoever helped.”**

This is not a rare configuration. It is the default. Registries — the institutions that hold the namespace, enforce signing, verify provenance, and respond to incidents — maintain no model of the humans. A package with one maintainer who last touched it in 2019 is, from the registry’s point of view, in the same state as a package with six active maintainers and a documented succession plan. The difference is invisible until it is catastrophic, and by then the registry’s only instrument is the incident-response hammer: yank the package, break the graph, hold a postmortem, change nothing.

The economics guarantee the problem grows. Ecosystem dependency graphs deepen every year; a large fraction of packages in every studied ecosystem report single-maintainer governance; maintainers are unpaid or barely paid, and burnout is the modal career outcome; dependency depth means a dormant utility can sit beneath a hundred thousand projects. The result is a supply chain whose *governance layer* operates entirely on folklore — Reddit threads titled “what do we do when the maintainer disappears” repeated annually, each answered from scratch.

This paper replaces the folklore with a protocol. The design constraint is stated up front, because it shapes everything: **the protocol must help without the power to force** — registries cannot compel maintainers, maintainers cannot compel successors, and any mechanism that centralizes the right to reassign packages creates an attacker’s bullseye worse than the disease. The instrument set is therefore opt-in per package, registry-facilitated, cryptographically enforced where it can be, and entirely public.

The thesis in one sentence: **maintainership continuity is the one property every software supply chain depends on and no registry governs, and it can be governed — opt-in, verifiable, and adversarially hard to abuse — with machinery that already exists.**

2 The evidence: five incidents, one shape

The incident record is not a list of unrelated surprises; read as governance events, they are the same event wearing different coats.

- **left-pad (2016).** A maintainer, in a naming dispute, unpublished 11 lines of code; within hours, builds across the JavaScript ecosystem failed en masse — the reachable dependency graph was damaged within hours because hundreds of thousands of packages (directly or transitively) depended on a single person’s morning mood. Postmortem: npm changed unpublish policy. Maintainer-succession state: unchanged since.
- **event-stream (2018).** The original author, no longer interested, transferred the package to an unknown volunteer who appeared helpful, with no community oversight; the new maintainer added an obfuscated dependency that targeted a Bitcoin wallet application (Copay) downstream. This is the exact shape of the xz attack, six years earlier, executed in months instead of years.
- **colors / faker (2022).** A long-suffering maintainer pushed an infinite-loop “protest” release of his own widely-depended-on packages, breaking CI pipelines everywhere within minutes. Lesson class: *live maintainers are also a risk*, and registries have no lane for “owner goes rogue” other than global remediation.
- **node-ipc (2022).** Protestware that destructively targeted users in specific countries. Same lesson, escalated: package maintainers can act on the installed base at will, and nothing in the distribution chain models intent.
- **xz-utils / CVE-2024-3094 (2024).** The full attack described in §1: multi-year trust accumulation, ownership transfer, backdoored release through legitimate channels. The community’s reconstruction (the maintainer’s own archived messages describe exhaustion and gratitude for help) documents the human mechanism in more detail than any prior incident.

Five coats, one shape: **the registry had no model of the maintainership, so the maintainership transitioned through private arrangements, and the blast radius was set by whoever ended up holding the keys.** Adjacent record: PyPI’s later critical-package 2FA mandate and organization accounts (2023), npm’s trusted publishers and provenance attestations (2023–2024), OpenSSF Scorecard — every one of these hardens *release integrity* (who can publish, is the artifact what was built) and none touches *continuity* (does publishing authority stay in trustworthy hands over years). The industry has been hardening the exact layer the incidents did not actually break.

3 The lifecycle model: abandonment as a state machine

The protocol’s first move is to make maintainer state *legible* — observable, timestamped, and queryable by dependents and the registry alike.

- **ACTIVE** — releases or merged substantive PRs within the package’s declared cadence window.
- **DECAYING** — cadence missed; the registry surfaces a public *staleness signal* (not a judgment; a timestamp).
- **DORMANT** — sustained inactivity beyond the package’s declared dormancy threshold, or bus factor drops to 1 with no declared successor. The package’s *continuity state* becomes a first-class badge in the registry UI and a structured field in the API.
- **ORPHANED** — DORMANT plus a failed liveness proof (dead-hand ping unanswered, owner unreachable, or maintainers explicitly walk away). The succession policy (§4) activates.

Two observations about the state machine. First, thresholds are *per-package declared*, not global: an exam-grade crypto library and a one-off CSS snippet have no business sharing a dormancy definition; defaulting policy per download-centrality band is the registry’s job. Second, transitions out of DORMANT are always

available to the incumbent (activity resumes, staleness clears) — the model describes attention, not virtue, and says so. Abandonment detection is not new (the academic ecosystem literature measures maintainer-activity decay, and truck-factor estimation is an established line) — the contribution is *operationalizing* it as registry-legible state with consequences attached, which no registry does.

4 The protocol

4.1 Succession policy as code

Each opt-in package carries a signed **succession manifest** in-repo and registry-visible:

```

succession:
cadence: 180d # ACTIVE window for this package's rhythm
dormancy: 400d # DORMANT threshold
quorum: # current maintainers with publish authority
- id: @maintainer-a
- id: @maintainer-b
transfer: # how authority moves (any subset may apply)
dead-hand: # pre-declared successor + public delay-lock
successor: @maintainer-b
delay: 14d # reversible window; published on trigger
community-takeover: # if DORMANT -> ORPHANED with no successor
sponsors: 3 # independent contributors vouching
review: 30d # public review window, security audit if central
elevation: # new-maintainer trust ladder (§4.3)
cosign-release-for: 90d
review-window: 7d # public review before any release lands

```

The manifest is the package’s *governance contract*, exactly as `Cargo.toml` is its build contract. Registries score it (a one-line OpenSSF Scorecard addition), dependents can query it in CI (“fail if any production dependency is DORMANT without policy”), and procurement can require it. Policy-as-code is what turns bus factor from an anecdote into an attribute.

4.2 The dormancy escalation ladder

When the registry’s state machine (§3) enters DORMANT, an automatic, entirely public sequence runs:

1. **T0 — Staleness surfacing.** Badge, API field, and a single structured notification to declared maintainers (liveness ping; see dead-hand, §4.4).
2. **T1 — AT-RISK flag.** After the declared threshold: the package page and API flag continuity risk; downstream CI policies may warn.
3. **T2 — Security hold (the narrow intervention).** For packages above a centrality band: *new publishes* enter a mandatory review window (default 7 days) before propagating. Note the precision: **installs of existing versions never pause**. The registry cannot and should not “freeze the ecosystem”; it can make a *transition of publishing authority* slow and visible. This is the difference between governance and hostage-taking.
4. **T3 — Succession execution.** Dead-hand transfer (§4.4) if declared; otherwise community takeover (§4.5); otherwise the package enters ORPHANED terminal state, graph intact, explicitly unlabeled for adoption.

Every step is on the public, append-only **continuity ledger** (§4.6): succession in open source becomes an auditable historical record, not archaeology.

4.3 Trust elevation: the anti-xz ladder

The xz attack’s mechanics dictate this component. When a *new* maintainer gains publish authority (by any path — transfer or takeover), the registry enforces a graduated release policy for a declared window (default 90 days):

- **Co-signature epoch.** Every release in the window requires signature by both the new maintainer *and* a pre-existing authority — an incumbent maintainer, a designated steward org (e.g., the ecosystem’s foundation, a Tidelift-style maintainer consortium), or the registry’s own attestation service. The event-stream hijack (transfer, then immediate malicious publish) fails here outright.
- **Delay-locked review.** Every publish in the window propagates after a public review window (default 7 days) — signed, timestamped, diff-visible on the continuity ledger. The xz backdoor shipped through an *ordinary* release that nobody had reason to slow down; here, the first releases of any newly-elevated maintainer are structurally slow and structurally public, which is exactly when years-long trust attacks are cheapest to interdict.
- **Attestation floor.** Builds must carry provenance attestations (the npm/PyPI machinery already deployed for trusted publishers) throughout the elevation window — the new maintainer’s releases are, by construction, tied to public build inputs.

After the window, the maintainer is fully trusted, exactly as today — the ladder is expensive only for the newcomer’s first 90 days, which is precisely the period in which every recorded trust-accumulation attack operated.

4.4 Dead-hand transfer

Maintainers can declare, today and while healthy, a successor with a **reversible delay-lock**: the transfer instruction is published on trigger (maintainer-initiated, or the registry’s liveness ping failing beyond a threshold), and executes after a public reversal window (default 14 days) during which the incumbent — or the successor, or a quorum — can cancel. This is dead-hand design from P-008, pointed at people instead of firmware: the liveness ledger, the trigger, and the recovery semantics are the same primitives. The maintainership version answers the modal real event, which is not death but *departure* — the burnout, the job change, the “I just don’t open these emails anymore.”

4.5 Community takeover

No successor declared, package ORPHANED, dependency centrality high: any community can file a takeover with *n* independent vouches (default 3 contributors with sustained history in the ecosystem — *tenure in the graph*, not real-world identity, deliberately), a public review window, and, above a centrality band, a third-party security audit of the first post-takeover release. The registry adjudicates only process, not merit. For the long tail of low-centrality packages, the same process runs with the audit waived — coverage for the tail, cost-proportional for the head.

4.6 The continuity ledger

A public append-only log of every succession-relevant event — state transitions, transfers, elevation windows, co-signatures, takeovers, reversals. Two consumers matter: dependents’ CI (a rule like “alert when a production dependency’s *maintainer set* changes,” which today is simply not expressible, is one query on the ledger) and researchers (the first population-scale dataset on maintainership succession, where today there is folklore and a handful of incident case studies).

5 Threat model and adversarial analysis

The patient attacker (xz replay). Multi-year trust accumulation, transfer, malicious release. Blocked by construction at the release layer: co-signature and delay-locked review apply exactly to a newly-elevated maintainer's first releases. Residual risk: the attacker *waits out the 90-day window before shipping* — the ladder raises the cost of the attack from “maintainer patience” to “maintainer patience + 90 days of clean releases under co-signature scrutiny + an established record,” and, critically, the *transfer event itself* is now public, which turns today's invisible private arrangement into a reviewable community signal. Honest bound: the ladder makes the attack slower and more visible, not impossible.

The sybil takeover. An attacker manufactures n vouches. Tenure-in-graph requirements (sustained contribution history across the ecosystem, not disposable accounts) make sybils expensive; centrality-band audits raise the price further for the packages worth attacking. This is a cost-imposition game — the design raises attacker cost by orders of magnitude at near-zero honest-user cost, which is the correct shape for supply-chain defense.

The malicious incumbent (colors/node-ipc). A *live, trusted* owner ships sabotage. The succession protocol does not and cannot prevent this class — authority is legitimate; the escalation ladder does not apply to established maintainers. Mitigation is architectural and honest: installs of *pinned* versions are unaffected; registry-level emergency yank (existing machinery) remains the response; and the continuity ledger at least makes the attacker's behavioral history (release cadence, co-maintainer abandonment — both were visible in the colors/node-ipc maintainers' records before the events) queryable by dependents who care to look. The paper scopes this honestly: **owner-goes-rogue is a different problem than owner-goes-away; the protocol solves the second and documents the first.**

Registry overreach. A registry abusing succession power to seize packages is an adversarial case the design takes seriously: all adjudication is process-only (was the review window observed? were vouches valid?), every event is on the public ledger, and the manifest is *opt-in* — a package with no succession policy has nothing for the registry to execute. Opt-in is simultaneously an adoption strategy and a check on institutional power.

Provenance bypass. All the ladder's teeth assume publishes flow through the registry's attested path; side-loading artifacts (vendor tarballs, mirrored repos) escapes it. The protocol's scope is the registry channel — where 99% of consumption flows — and says so.

6 Why nothing like this exists (yet)

The state was illegible. Registries cannot govern what they do not model, and maintainer activity was (until the state machine) private arithmetic — GitHub events, release timestamps, no registry semantics. Scorecard measures repo hygiene; no instrument measures *continuity*.

The autonomy norm. Open source’s deep cultural norm — maintainer sovereignty over their package — reads any registry involvement as appropriation. The design’s opt-in posture, process-only adjudication, and reversible windows are the direct response; the norm is respected structurally, not rhetorically. The Linux kernel’s maintainership hierarchy is the culture’s own existence proof that layered succession is native to open source when scale demands it — the protocol productizes what the kernel practices informally.

Incidents were misfiled. Every postmortem in §2 was filed under security (harden the release channel) or distribution (change unpublish rules), because those are the layers registries own. The succession layer sat between jurisdictions — the maintainer’s private choice, the attacker’s social engineering, the community’s folk process — and between-jurisdiction problems stay unsolved until someone writes the missing layer down. That is this paper’s move.

No one was paid to. Continuity is a public good inside a commons of unpaid maintainers; the funding layer (Tidelift, GitHub Sponsors) pays for *maintenance*, not for *transition*. The dead-hand and manifest instruments are cheap (an afternoon per package) but someone had to design them; nobody’s job description contained it.

7 Evaluation design

Historical replay. The decisive experiment is counterfactual replay against the incident record: implement the state machine and ladder; replay npm/PyPI/crates.io event histories (public datasets exist for release and maintainer-activity timelines); ask four questions per incident — (a) would the state machine have flagged the package’s continuity state before the event? (b) would the elevation ladder have applied to the eventual attacker? (c) would the delay-locked review have surfaced the malicious change within its window? (d) what false-positive load (packages entering DORMANT with healthy outcomes) would the default thresholds have produced across the ecosystem? The last question is the calibration constraint: a ladder that fires on 20% of the ecosystem weekly is noise; defaults must be tuned so that DORMANT flags are rare enough to carry information.

Agent-based ecosystem simulation. Synthetic ecosystems with realistic degree distributions (heavy-tailed, matching published ecosystem-graph measurements), maintainer-attention decay processes, and adversarial strategies (patient attacker, sybil ring, rogue incumbent); measure compromise rate and ecosystem-breakage rate under protocol-on vs. protocol-off, plus the cost overhead on honest maintainers (target: < 1 hour/year/package).

Migration feasibility. Pilot on one registry (crates.io is the cleanest candidate: smallest ecosystem among majors, strongest policy culture, explicit documented transfer practice to build on); measure opt-in rates under a Scorecard credit incentive; run one live community takeover under full protocol as the reference case.

Ledger utility. Deploy the “alert on maintainer-set change of any production dependency” rule against a real dependency graph; measure alert precision over a quarter (what fraction of maintainer-set changes correlate with meaningful events — abandonment, hijack, migration).

8 Objections, confronted

“Maintainer autonomy is sacred.” Agreed, and structurally honored: opt-in, reversible, process-only. A maintainer who declares *no* policy is exactly as sovereign as today — the protocol merely exists for those who want their packages to outlive their attention, which (see: every “looking for maintainer” issue ever filed) is most of them eventually.

“False positives will freeze packages.” The security-hold pauses *new publishes*, never installs; a false DORMANT flag costs a healthy maintainer one ping and one click. The system’s failure mode is deliberately boring.

“Attackers will just wait out the 90 days.” Some will; the attack cost rises from social-engineering one tired human to sustaining 90 days of co-signed, publicly-reviewed releases with an established successor — while the *transfer event itself* is published on the ledger for anyone’s CI to flag. Deterrence by cost and visibility, not by impossibility; §5 is explicit about this bound.

“Registries don’t want the liability.” Registries already adjudicate unpublish disputes, name squats, and hijack emergencies — the messiest, least-structured versions of this jurisdiction. A published protocol with opt-in scope is a liability *reduction* relative to improvised case-by-case discretion.

“Just fork it.” The fork answer misunderstands the asset: the *name* is the trust channel. Consumers resolve event-stream, not a content hash; a fork without the name re-fights the discovery battle the registry exists to settle. Succession governance is name governance; that is precisely why it must live at the registry layer.

“This is a registry grab for power.” The counter-institutional design is the answer: public ledger, process-only adjudication, opt-in, reversal windows, and the Scorecard *credit* (not penalty) framing for declaring policy. The protocol’s power gradient runs toward dependents and communities, not toward the registry.

9 Limitations

Coverage is opt-in, so the vulnerable installed base of unpolicied packages decays only as fast as culture changes; the ladder’s window is calibrated for npm-scale cadences and may misfit slow-release ecosystems (kernel-adjacent, scientific); tenure-in-graph sybil resistance is vulnerable to long-con identity operations by patient, well-resourced attackers (the bound stated in §5); rogue-incumbent sabotage remains out of scope; and registry adoption is a political variable the design can only make easy, not inevitable. None of these bound the specification’s correctness; they bound its deployment rate.

10 Conclusion

The software supply chain has spent a decade hardening the layer the incidents exposed least — artifact integrity — while the layer every postmortem actually walked through — a tired maintainer, a stranger, a transfer nobody recorded — remained folklore. xz was not a hack that happened to a healthy system; it was the healthy-looking surface of a system with no succession layer at all. This paper specifies that layer: state that is legible, policy that is code, transitions that are slow and public when they should be, and a ledger that turns “who maintains this” from archaeology into audit. The machinery exists. The incidents wrote the requirements. What was missing was the layer written down — so it is, now.

References

1. CVE-2024-3094 (xz-utils/liblzma backdoor) and community postmortems (March 2024). Primary reconstruction: the xz backdoor timeline as documented in OSS-Security mailing list threads and subsequent analyses. [Incident record; high confidence; detail verification queued.]
2. left-pad incident (March 2016). Reporting: *The Register*, *The Guardian* (“How one man’s code holds his city hostage”? — exact headlines vary; the event and 11%-of-ecosystem framing are standard). [Incident record; event high confidence, headline framing queued.]
3. event-stream hijack (November 2018) targeting Copay/BITBOX wallets. Reporting: *ZDNet* / GitHub issue #110 on event-stream. [Incident record; high confidence.]
4. colors and faker protest releases (February 2022). Reporting: *The Verge*, *Ars Technica*. [Incident record; high confidence.]
5. node-ipc protestware (March 2022). Reporting: *BleepingComputer*, *Ars Technica*. [Incident record; high confidence.]
6. PyPI: 2FA mandate for critical projects and organization accounts (2023–2024). [Registry policy record; high confidence.]
7. npm trusted publishers and provenance attestations (2023–2024). [Registry policy record; high confidence.]
8. OpenSSF Scorecard — automated repository-security grading; no continuity criteria as of 2024. [Registry-adjacent record; high confidence on the absence claim, re-verifiable in one query.]
9. Avelino, G., et al. (2016–2017). “A systematic approach to multiple truck factor estimation” / truck factor literature. *International Conference on Program Comprehension* (2016). [Peer-reviewed; high confidence on the line’s existence.]
10. Ecosystem abandonment and maintainer-activity decay studies (GitHub abandonment prediction literature, 2015–2019; e.g., work measuring dormant project rates across ecosystems). [Academic line; PARTIAL confidence on specific instances; verification queued.]
11. Linux kernel maintainership hierarchy — documented subsystem maintainer model with layered review/approval authority. [Practice record; high confidence.]
12. Tidelift — maintainer funding subscriptions (2018–). [Industry record; high confidence.]
13. crates.io package-transfer practice documentation (Rust ecosystem). [Registry documentation record; moderate-high confidence; verification queued.]
14. p-rick P-008, *The Afterlife of Devices* (this program, 2026): the shared dead-hand/trigger/ledger primitives, pointed at firmware escrow instead of maintainership. [Program cross-reference.]
15. p-rick P-004, *Degradation Contracts* (this program, 2026): hysteresis semantics reused for state-machine transitions (flap-resistant staleness signals). [Program cross-reference.]
16. Software Heritage — universal source-code archive (UNESCO-partnered); preserves artifacts, not maintainership continuity. [Infrastructure record; high confidence; used as the “what preservation exists today” contrast in P-011.]

Working-paper note: references marked “verification queued” are recorded from domain knowledge at high confidence and await the program’s live citation-verification pass (see agents.md); the verification ledger is maintained with the paper sources.